

Hydro Model Architectures

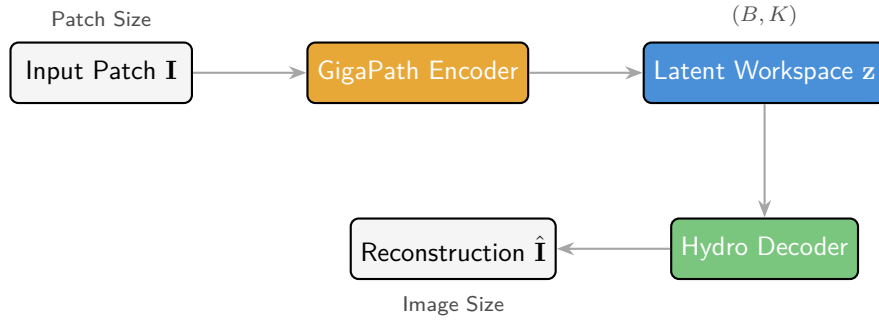
autopath

1 Overview

The **Hydro** datablock provides a flexible framework for latent-to-image decoders. It is designed to invert feature extractors (e.g., GigaPath) by mapping their embeddings back to the original image space.

1.1 Large-scale System Architecture

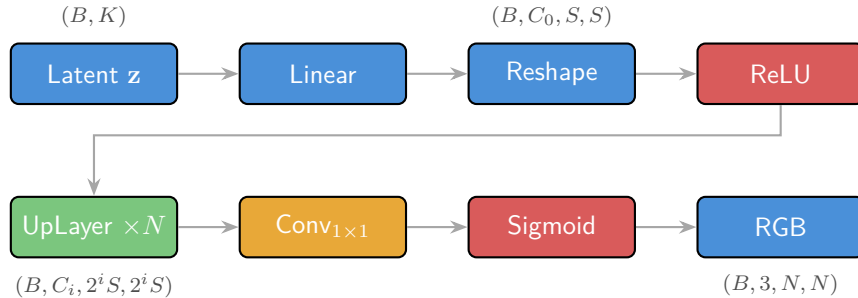
In the context of the overall pipeline, **Hydro** serves as the generative component that transforms compressed semantic embeddings into spatial reconstructions. The global latent vector $\mathbf{z}$ is the primary interface between the large-scale encoder and the decoder.



The latent vector $\mathbf{z}$ is produced by a fixed (or fine-tuned) backbone that summarizes the semantic content of a histology slide or patch. This vector provides the global context used by **Hydro** features like Feature Modulation to inform the progressive upsampling process.

2 Model_0: Deconvolutional Decoder

Model_0 is the baseline architecture, using progressive upsampling to reconstruct $N \times N$ RGB images from K -dimensional latent vectors.



2.1 Architectural Improvements to Combat Blur

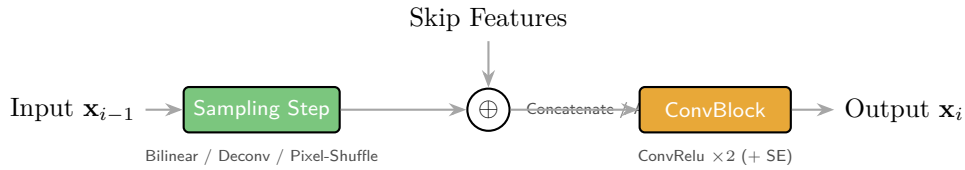
To address the blurriness observed in `Model_0` outputs, several optional components can be enabled. These features prioritize the preservation and enhancement of high-frequency information and are situated within the progressive upsampling loop.

2.1.1 Overview of Improved Upsampling Path

where $\mathbf{x}_i \in \mathbb{R}^{C_i \times H_i \times W_i}$.

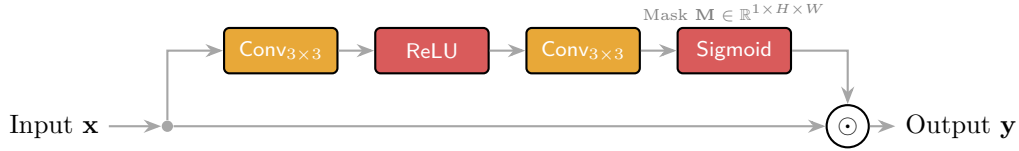
2.1.2 Internal Architecture of UpLayer

The `UpLayer` is the fundamental building block of the CNN decoder. It handles spatial upsampling, optional skip connections from an encoder, and local feature refinement.



2.1.3 Spatial Attention Gates (SAG)

The `SpatialAttentionGate` identifies and emphasizes salient spatial features (e.g., edges and textures). It is situated at the end of each upsampling stage to refine the newly generated spatial details.



The gate receives the upsampled feature map $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$. In its enhanced version, it employs 3×3 convolutions to capture better spatial context when computing the mask, effectively identifying sharp transitions like edges and textures which are then selectively amplified.

The gate receives the upsampled feature map $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$. While the gate itself employs 1×1 (pointwise) convolutions, it effectively learns to identify spatial transitions by interpreting the local neighborhood information that has already been “collapsed” into the channel dimension by the preceding 3×3 convolutions in the `UpLayer`.

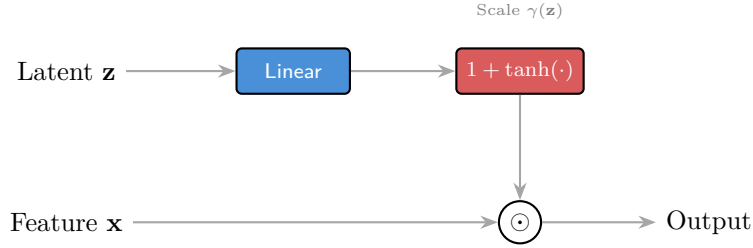
The mask generator sub-network learns a **pixelwise** spatial mask $\mathbf{M}$ where each value $\in [0, 1]$. This single-channel mask is shared across all C channels. The output is computed via the Hadamard product (element-wise multiplication $\odot$) with broadcasting:

$$\mathbf{y}_{c,h,w} = \mathbf{x}_{c,h,w} \times \mathbf{M}_{1,h,w}$$

By focusing representational capacity on these spatially-encoded transitions, SAG effectively reduces the smoothing artifacts.

2.1.4 Feature Modulation

Feature modulation allows the global latent vector $\mathbf{z}$ to dynamically scale the intensity of intermediate feature maps. This provides a mechanism for the latent representation to globally influence spatial details at every upsampling stage.



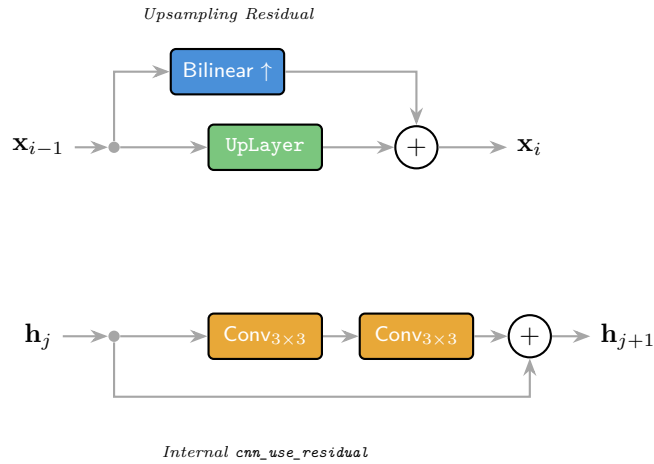
This global conditioning ensures that the reconstructed image maintains sharpening consistency across different scales, as the latent vector can “boost” specific frequency components needed for a sharp reconstruction.

2.1.5 Residual Architectures

Two distinct types of residual connections are available to improve gradient flow and preserve signal fidelity.

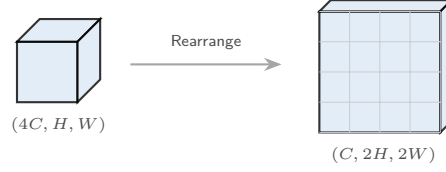
Upsampling Residuals (`cnn_use_residual_upsampling`) This resolution-skipping connection operates **across** an upsampling block. It provides a direct path for low-frequency content by upsampling the previous stage’s feature map (typically via bilinear interpolation) and adding it to the current stage’s output.

Internal Residuals (`cnn_use_residual`) This connection operates **within** a single ConvBlock at a constant resolution. It adds the block’s input directly to its output ($y = f(x) + x$), helping gradients propagate through deep stacks of convolutions without attenuation or blurring.



2.1.6 Pixel-Shuffle Upsampling (cnn_use_pixel_shuffle)

Instead of using bilinear interpolation or standard transposed convolutions (deconvolution), the model can be configured to use **Pixel-Shuffle** (Depth-to-Space) upsampling. This technique rearranges channels into higher spatial resolution ($2\times \uparrow$), which often results in significantly sharper edges as it avoids the interpolation artifacts typical of standard upsampling kernels.

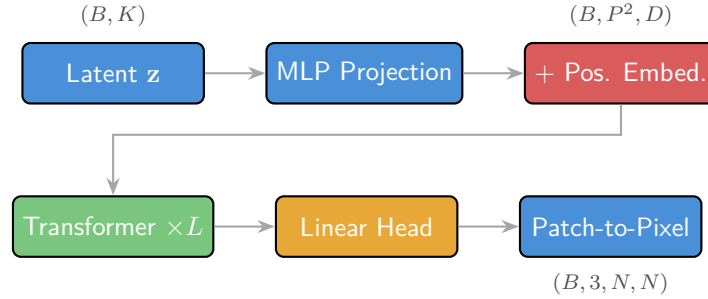


3 Model_1: Vision Transformer (ViT) Decoder

Model_1 adopts a transformer-based architecture to model long-range dependencies and produce sharper reconstructions by operating on a tokenized representation of the image.

3.1 Architecture Overview

The architecture follows a “latent token to patch” approach:



3.1.1 Tokenization and Projection

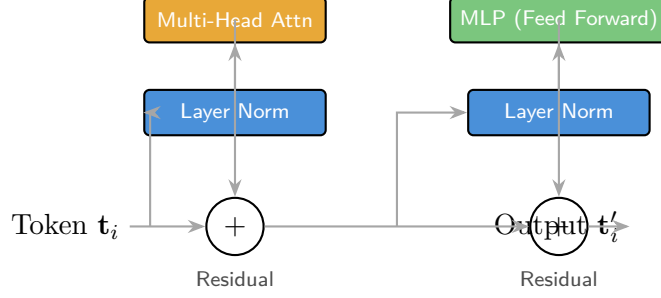
The latent vector $\mathbf{z}$ is projected into a sequence of $M = (N/p)^2$ tokens, where p is the patch size (e.g., 16).

$$\mathbf{z} \xrightarrow{\text{MLP}} \mathbf{T} \in \mathbb{R}^{M \times D}$$

Learnable 2D positional embeddings are added to provide spatial context to the transformer.

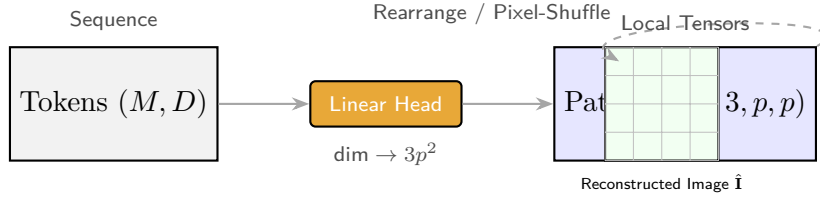
3.1.2 Transformer Backbone

A series of L standard Transformer blocks (Multi-Head Self-Attention + MLP) process the token sequence. This allows information to propagate globally across the image grid.



3.1.3 Reconstruction Head and Patch-to-Pixel

Each token $\mathbf{t}_i \in \mathbb{R}^D$ is projected to a patch $\mathbf{P}_i \in \mathbb{R}^{3 \times p \times p}$ using a linear layer. The patches are then rearranged to form the final image $\mathbf{I} \in \mathbb{R}^{3 \times N \times N}$.



4 System Configuration

The Hydro datablock uses a shared configuration dataclass:

Parameter	Default	Description
latent_dim	1536	Input latent vector dimension K
image_size	256	Output resolution $N \times N$
hidden_channels	256	Base channel count (Model_0) / Dimension D (Model_1)
patch_size	16	Patch size for Model_1
num_layers	12	Number of layers (Transformer blocks) for Model_1
version	0	Model architecture version (0 or 1)
use_residual_upsampling	False	Enable residual skip connections in Model_0
use_spatial_attention_gates	False	Enable spatial attention in Model_0
use_feature_modulation	False	Enable latent-based feature modulation in Model_0
loss_type	mse	MSE, L1, L0, or LPIPS reconstruction loss

5 Loss Functions

Reconstruction quality is improved by incorporating perceptual metrics:

$$\mathcal{L} = \begin{cases} \frac{1}{3N^2} \|\hat{\mathbf{I}} - \mathbf{I}^*\|_2^2 & \text{if loss_type = "mse"} \\ \frac{1}{3N^2} \|\hat{\mathbf{I}} - \mathbf{I}^*\|_1 & \text{if loss_type = "l1"} \\ \text{LPIPS}(\hat{\mathbf{I}}/255, \mathbf{I}^*/255) & \text{if loss_type = "lpips"} \\ \|\nabla \hat{\mathbf{I}} - \nabla \mathbf{I}^*\|_1 & \text{if loss_type = "grad"} \end{cases}$$

The **LPIPS** (Learned Perceptual Image Patch Similarity) loss compares deep features from a pre-trained network. Crucially, as the network assumes $[0, 1]$ normalized inputs, the model's $[0, 255]$ outputs are scaled by $1/255$ prior to loss computation to ensure the perceptual weights are applied correctly.

The **Gradient Loss** (grad) explicitly penalizes differences in the spatial derivative of the image, encouraging the model to match the sharp edges of the target rather than just the average pixel intensities.